Stack Tecnológico — Replatforming `ATPPuestos`

> Documento de referencia del stack y de la metodología con la que se obtuvo la información del proyecto legacy (GeneXus 18) y del proyecto actual (monorepo Java/Spring Boot). Audiencia: equipo de replatforming y nuevos integrantes.
> Fuentes: `docs/analisis-ATPPuestos.md`, `docs/design.md`, `docs/genexus18mcp.md`, `openspec/constitution/*`, `.opencode/*`, `pom.xml`.

---

1. Propósito

Este documento responde tres preguntas:

1. **¿Qué stack usamos y por qué?**
2. **¿Cómo obtuvimos la información del sistema anterior (GeneXus)?**
3. **¿Cómo obtuvimos la información del sistema actual y cómo la gobernamos?**

Y deja explícito el **arnés de control** (OpenCode: agentes y subagentes) y el flujo **Spec-Driven Development (SDD)** que usamos para producir las especificaciones y el código.

2. Stack objetivo (resumen)

Capa	Tecnología	Obligatorio
Lenguaje	Java 17 (LTS)	Sí
Framework	Spring Boot 3.5.16	Sí
Presentación	Spring MVC + Thymeleaf	Sí
JavaScript	Puntual, vanilla, sin framework ni build	Sí
Persistencia	Spring Data JPA / Hibernate 6.6	Sí
Validación	Jakarta Bean Validation	Sí
DTOs	Java `record` inmutables	Sí

Capa	Tecnología	Obligatorio
Base de datos	SQL Server existente (`Rentas`), sin rediseño	Sí
Build	Maven Wrapper multi-módulo	Sí
Pruebas	JUnit 5 + Mockito + MockMvc + IT contra BD real	Sí
Empaquetado	Monolito modular (1 fat JAR)	Sí

Regla de oro de datos: la BD es existente; se **mapea**, no se rediseña. `ddl-auto=none`, sin DDL, sin migraciones, sin Flyway/Liquibase, sin `schema.sql`/`data.sql`. Los nombres físicos legacy (tabla de 8 chars, columna de 10 chars) se preservan con `@Table(name=...)` y `@Column(name=...)`.

3. Arquitectura del monorepo

```
None
/
├── pom.xml                      ← parent (Spring Boot 3.5.16) + Maven Wrapper
├── atp-nucleo-compartido/       ← catálogo + personas + base de seguridad + API
REST /api/v1
├── atp-puestos-camineros/       ← artefacto desplegable (login + dominio + vistas
Thymeleaf)
├── openspec/                    ← especificaciones formales (EARS, ISO/IEC 25030)
├── docs/                        ← análisis, diseño, Postman
├── .opencode/                   ← arnés de agentes y comandos
└── .engram/                     ← memoria persistente del agente
```

Regla de dependencia: `web → service → repository → domain`. `domain` no depende de nada; `shared` es transversal. **Prohibido** el atajo `web → repository` y `domain → web`.

Dos canales de respuesta: `/api/**` devuelve JSON con cuerpo uniforme `{code, message}`; el resto de las rutas devuelve HTML Thymeleaf. El `GlobalExceptionHandler` negocia según el prefijo de ruta. Ambos canales consumen **los mismos servicios**: la lógica de negocio vive una sola vez en `service/**`.

4. Cómo se obtuvo la información del proyecto anterior (GeneXus)

4.1 La KB legacy

Elemento	Valor
KB (origen, solo lectura)	`F:\Modelo\Gx18\ATPPuestos` (`ATPPuestos.gxw`)
GeneXus	18.0.13 (U13) — `C:\Program Files (x86)\GeneXus\GeneXus18`
Ambiente activo	`Gen18` — generador **Default (Java)** — web path `Data007\web`
BD de metadatos KB	`GX_KB_ATPPuestos`
BD de la aplicación	SQL Server
Tamaño KB en disco	6.738 archivos / ~513 MB
Índice MCP	1.615 entradas (objetos + atributos + clases)
Inventario	40 tablas, 32 transacciones, 77 procedimientos, 69 WebPanels, 6 DataProviders

Hallazgo crítico de paridad: los nombres **físicos** de tabla son heredados (muchos de 8 caracteres, era GX15) mientras las transacciones ya usan nombres largos. El mapeo `Tabla física ↔ Transacción` no es 1:1 trivial (p. ej. `Provincia → PROVINCI`, `Localidades → Localida`, `Productos → PRODUCTO`/`PRODUCT1`).

4.2 Canal de introspección: servidor MCP `genexus18mcp`

genexus-mcp (Node.js) expone la API/SDK de GeneXus 18 como **54 tools** para leer la KB sin abrir el IDE.

- **Versión fijada:** `genexus-mcp@3.9.2` (no `@latest`, para reproducibilidad).
- **Runtime:** `npx.cmd` en Windows (no `npx`).
- **Configuración** en `C:\Users\walterg20\.config\opencode\opencode.jsonc`:

```jsonc
"genexus18mcp": {
  "type": "local",
  "command": ["npx.cmd", "-y", "genexus-mcp@3.9.2"],
```

```
  "enabled": true,
  "environment": {
    "GX_CONFIG_PATH": "F:\\Modelo\\Gx18\\ATPPuestos\\config.json"
  }
}
```

`GX_CONFIG_PATH` apunta al `config.json` **de la carpeta de la KB**, no al de la instalación de GeneXus: es el que le dice al SDK a qué KB apuntar.

Tools usadas (lectura): `genexus_list_objects`, `genexus_read` (parts `Structure`, `Rules`, `Events`, `Source`), `genexus_structure` (`get_indexes` → nombre físico de tabla/PK/índices), `genexus_search_source`, `genexus_query`, `genexus_module`, `genexus_analyze` (`impact`/`dependencies`/`context`), `genexus_kb`, `genexus_whoami`, `genexus_doctor`.

Tools prohibidas sobre la KB legacy: toda tool de escritura/compilación/creación (`genexus_create`, `genexus_edit`, `genexus_edit_and_build`, `genexus_delete_object`, `genexus_apply_pattern`, `genexus_refactor`, `genexus_deploy`, `genexus_run_object`, …). La KB es **fuente de verdad inmutable** (G1).

4.3 Puerta G-DATA

La KB describe la estructura **lógica**; la BD real describe la **física** y se contrastó contra `INFORMATION_SCHEMA.COLUMNS`, `KEY_COLUMN_USAGE` y `REFERENTIAL_CONSTRAINTS`. Eso reveló:

- Truncado de nombres a **8 chars** (tabla) y **10 chars** (columna): `FerDescripcion → FerDescrip`.
- `CHARACTER(n)` es `char(n)` fijo (rellena con espacios → `trim` al leer), no `varchar`.
- `DATE`/`DATETIME` reales donde la KB dice `DATE` (`Feriados.FerFecha`, `Product1.ProdFchVig`).
- `NUMERIC` que mapea a `decimal` (`Areas.AreaId` → `BigDecimal`).
- **Columnas huérfanas** (existen en la BD, no en la KB): `Areas.AreaBaja/AreaEmails/AreaAdjun/AreaRespons`, `CALLES.CallAlt1/2/3`, `Localida.LocCodAfip`, `PROVINCI.ProvCodAfip`, `PRODUCTO.ProdReppa`, `PRODUCT1.ProdImpu5/ProdFeLog/ProdUsu`.
- Sin `IDENTITY` ni `COLUMN_DEFAULT` en las tablas → la PK viaja en el body del `POST`.
- Sin FK física `Localida → PROVINCI` (asociación solo lógica).

Detalle completo: `docs/analisis-ATPPuestos.md` §13 y §15.

4.4 Reglas de introspección aprendidas

- **Colisión de nombres:** existe la carpeta `Feriados` y la transacción `Feriados`. Siempre pasar `type: 'Transaction'` explícito.
- **El MCP no expone nombres físicos de columna** (sólo de tabla vía `get_indexes`). De ahí el supuesto ASM-01, validado en G-DATA.
- **Transactions sin reglas:** muchas maestras tienen `Rules`/`Events` vacíos. No asumir lógica de negocio donde no la hay; verificar con `genexus_read`.
- **Arranque en frío:** la primera tool de una sesión puede tardar (descarga `npx` + SDK). Si falla, verificar GeneXus instalado/licenciado y `GX_CONFIG_PATH`.
- **GeneXus 15 no es analizable** con esta herramienta (MCP soporta 16/17/18). Requiere exportación `.xpz`/`.xml`. Hay 6 KB de GX15 en el entorno que no pudieron contrastarse.
- **KB en construcción:** si una búsqueda vuelve vacía, esperar con `genexus_lifecycle action=status wait=30 freshness=current` y reintentar.

4.5 Regla de oro

> La KB GeneXus es **SOLO LECTURA**. Prohibido modificar, compilar (`Build All`), regenerar (`Reorganize`) o eliminar objetos/tablas. Abrir/seleccionar/leer no altera la KB. Esta regla es absoluta (G1) y aplica a humanos y a subagentes.

5. Cómo se obtuvo la información del proyecto actual

- **Código del monorepo:** módulos, paquetes, entidades JPA, repositorios, servicios, controllers REST/MVC, plantillas Thymeleaf, CSS de tokens, JS puntual.
- **Configuración:** `pom.xml` (parent Spring Boot 3.5.16, Java 17), `application.yml` (único, en el núcleo), `.env.example`, `openspec/config.yaml`.
- **Constitución:** `openspec/constitution/stack.md`, `conventions.md`, `sdd-workflow.md` (reglas no negociables del stack y del trabajo).
- **Análisis de diseño:** `ATPEvo2.css` (3.754 líneas) → tokens CSS documentados en `docs/design.md` (paleta, tipografía, espaciado, mapeo legacy→componente, JS puntual permitido).
- **Parque de aplicaciones (§14 del análisis):** 3 KB de Gx18 comparten la misma instancia SQL Server. `ATPPadronWeb` es un proxy thin (18 tablas de proceso). Los catálogos están **duplicados** entre bases → el "núcleo compartido" del replatform es **código** (biblioteca Maven), no datos. `ATPWebGX18` (Gestión ATP, 10.867 objetos) y el Portal Contribuyente quedan **fuera** de este repo.

6. Arnés de control — OpenCode

El arnés vive en `.opencode/` y en `agents.md` (raíz). OpenCode es el entorno que orquesta agentes y subagentes, y el que expone los MCPs.

6.1 Agentes

Agente	Modo	Rol
`coordinador`	all	Orquesta el flujo SDD. **No escribe código de aplicación.** Solo puede escribir en `openspec/features/**` y Engram. Lanza subagentes.
`analista-gx`	subagent	Ingeniería inversa GeneXus 15/18 → specs OpenSpec/EARS. Solo lectura sobre la KB.
`implementador-backend`	subagent	Capa Java/Spring Boot (entidades JPA, repositorios, servicios, seguridad, API REST).
`implementador-vistas`	subagent	Capa Thymeleaf (vistas, fragmentos, CSS de tokens, JS puntual) y controllers MVC.
`verificador-qa`	subagent	Verifica con evidencia: suites, paridad funcional, paridad de BD, seguridad, gobernanza.
`explore`	subagent	Reconocimiento barato del repo.

El coordinador **sólo** puede lanzar: `analista-gx`, `implementador-backend`, `implementador-vistas`, `verificador-qa` y `explore`. No se lanza a sí mismo ni a otros agentes.

6.2 Comandos

Comando	Uso
`build-feature`	Construcción de una feature aprobada.
`extract-spec`	Extracción de spec desde la KB.

Comando	Uso
`verify-quality`	Verificación de calidad.

6.3 Permisos (guardarraíles del arnés)

- El coordinador tiene `edit` denegado globalmente, excepto `openspec/features/**`.
- El analista tiene `edit` denegado globalmente, excepto `openspec/features/**`.
- Ningún subagente puede escribir en la KB legacy ni en `docs/` (salvo lo que el coordinador autorice explícitamente).
- Los dos implementadores escriben en el mismo árbol Maven pero en rutas **disjuntas** (`web/rest` + dominio el backend; `web/mvc` + `templates` + `static` el de vistas), por lo que pueden correr en paralelo sin colisión.

6.4 Memoria persistente (Engram)

- `.engram/` es portable y trackeable.
- `mem_save` se usa tras cada decisión, bugfix, descubrimiento o convención.
- `mem_session_summary` se usa al cerrar la sesión (goal, instructions, discoveries, accomplished, next steps, relevant files).
- `mem_context` / `mem_search` se usan al iniciar o cuando el usuario pide recordar.

6.5 MCPs disponibles

MCP	Uso
`genexus18mcp`	Introspección de la KB legacy (solo lectura).
`context7`	Documentación actualizada de librerías/frameworks/SDKs.
`engram`	Memoria persistente.
`figma`	Diseño (cuando aplique).
`chrome-devtools`	Depuración de UI en navegador.
`codegraph`	Grafo de código (cuando el repo esté indexado).

7. Spec-Driven Development (SDD)

7.1 Norma y sintaxis

- **Norma:** ISO/IEC 25030 (paridad funcional por comportamiento observable).
- **Sintaxis de requisitos:** EARS (Ubiquo, dirigido por evento, dirigido por estado, comportamiento no deseado, opcional). Cada requisito tiene id estable (`R-<NNN>` o `R-<MODULO>-NNN`) y criterio de verificación.
- **Documentación y specs en español.** Código (identificadores), nombres de tabla/columna y rutas técnicas en inglés o nombre físico heredado.

7.2 Flujo obligatorio

1. **Reconocimiento** — leer `agents.md`, `openspec/constitution/`, `openspec/`; consultar Engram.
2. **Especificar** — `analista-gx` produce `spec.md` (EARS), `plan.md` (paridad JPA + contratos REST
    + contrato de vistas) y `tasks.md` (tareas atómicas `T-NNN`).
3. **Gate 1 — validar la spec** — sintaxis EARS, verbos normativos, paridad 1:1 de tablas/columnas, ausencia de DDL, tareas atómicas y verificables. Si falla, volver a `analista-gx`.
4. **Implementar** — `implementador-backend` e `implementador-vistas` en paralelo, dentro de sus rutas.
5. **Gate 2 — verificar** — `verificador-qa` corre unitarios, ITs de lectura y pruebas de vistas (MockMvc), y reporta con evidencia (rutas de log).
6. **Cerrar** — marcar ítems en `tasks.md` sólo si hay evidencia verde; lo pendiente queda explícito.
7. **Persistir** — `mem_save` de cada decisión/hallazgo/convención nueva y `mem_session_summary` al cerrar la sesión.

7.3 Módulos de spec actuales

Módulo	Alcance	Estado
`00-alcance`	Alcance migrable in/out (incluye lo OUT del monorepo)	✅
`01-nucleo-datos`	Núcleo de datos ya implementado (portado)	✅
`02-abm-productos`	ABM de `PRODUCTO` + `PRODUCT1` (vigencias) en Thymeleaf	⏳

Módulo	Alcance	Estado
`03-login`	Login completo de Puestos Camineros	⏳

7.4 Guardarraíles (G1–G7)

#	Regla
G1	KB GeneXus = **SOLO LECTURA**.
G2	**Spec-Driven Development**: sin spec aprobada, no hay código.
G3	**Sin secretos en el repositorio** (sólo variables de entorno).
G4	**Archivos de configuración < 500 líneas.**
G5	**Sin DDL ni migraciones.**
G6	**JavaScript puntual** (vanilla, sin framework, sin bundler, sin CDN, sin `npm` en el build).
G7	**Monolito modular** (regla de dependencia `web → service → repository → domain`).

7.5 G-POSTMAN

Toda agregación, modificación o eliminación de endpoint debe reflejarse en `docs/postman/ATPPuestos-API.postman_collection.json` en el mismo commit. Regla obligatoria de mantenimiento (`docs/postman/README.md`).

8. Decisiones clave y desvíos documentados

- **Capa de presentación:** cambió de Angular a **Thymeleaf** (decisión del equipo, 2026-10-22). El resto del alcance no cambió.
- **Color de error:** `#C0392B` en lugar de `#FF0000` (contraste AA).
- **Texto secundario:** `#555555` (5,77:1 sobre blanco) en lugar de `#747474` (3,61:1, bajo AA).
- **Color de éxito:** `#2E7D32` (el legacy no define uno).
- **Dialecto Hibernate:** `SQLServer2012Dialect` (v11, mínima soportada) para eliminar HHH000511.
- **PK:** viaja en el body del `POST` (no hay `IDENTITY` en las tablas legacy).

- **Cookie de sesión:** `SameSite=Lax` como mitigación para la ausencia de CSRF en `/api/**`.
- **Núcleo compartido:** es **código** (biblioteca Maven), no datos (los catálogos están duplicados entre bases en el legacy).
- **Planes de Pago / Portal Contribuyente / Gestión ATP:** fuera de alcance de este repo; registrados en `openspec/features/00-alcance/`.

9. Referencias cruzadas

- Diagnóstico F0 + G-DATA + mapa del parque: `docs/analisis-ATPPuestos.md`
- Sistema de diseño: `docs/design.md`
- Genexus MCP (instalación y gotchas): `docs/genexus18mcp.md`
- MCP GeneXus (configuración): `docs/mcp-genexus.md`
- DevOps (herramientas y versiones): `docs/devops.md`
- Constitución: `openspec/constitution/stack.md`, `conventions.md`, `sdd-workflow.md`
- Specs: `openspec/features/00-alcance/`, `01-nucleo-datos/`, `02-abm-productos/`, `03-login/`
- Arnés: `agents.md`, `.opencode/agents/`, `.opencode/commands/`
- Configuración OpenSpec: `openspec/config.yaml`
- Plantilla de entorno: `.env.example`
- Colección Postman: `docs/postman/ATPPuestos-API.postman_collection.json`

Generado en F1 — 2026. La KB original se conserva intacta como fuente de verdad.